

Managing I/O Operations

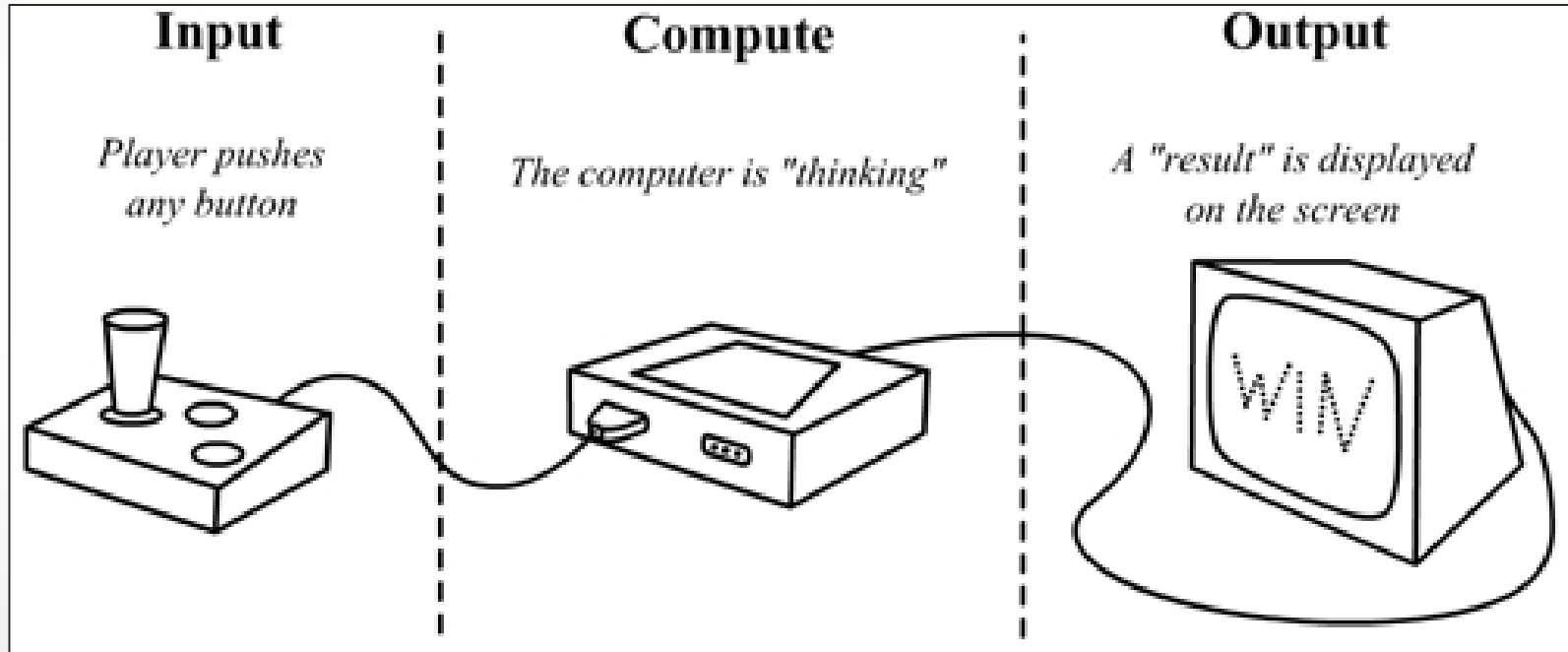
Subject: Programming for Problem Solving

Prepared By: Kunal J Sahitya

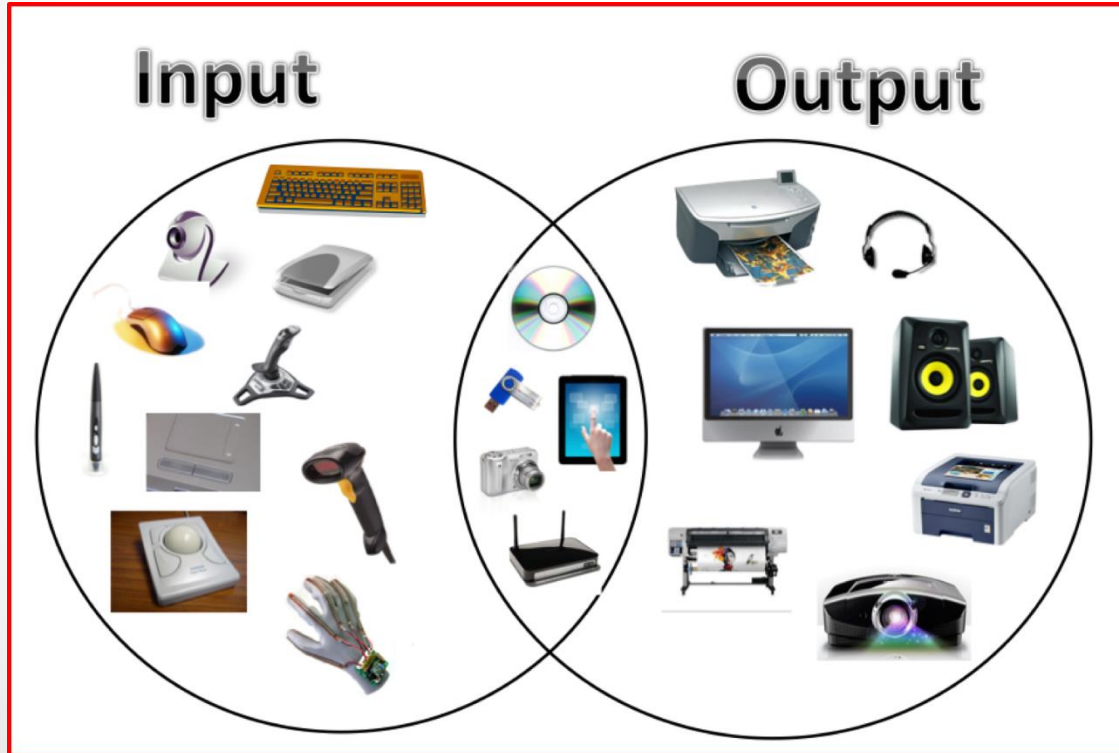
Introduction

- Input / output (I/O) is an essential part of any computer program for interaction of user and machine.
- Most of the higher level languages support built-in I/O functions.
- C has I/O functions as part of its library which mainly include “**stdio.h**” and “**conio.h**” header files.

I/O Operation



Supporting Devices



Reading a Character

- To read a single character input from user 'getchar()' function is used.
- Syntax:

```
var_name = getchar();
```

- E.g:

```
char user_input;  
user_input=getchar();
```

Character Test Functions

Function	Description	Example
isalnum()	This function is used to check whether a character is an alphanumeric(i.e A to Z or a to z) or (0 to 9).	isalnum('a')=True(True means non zero value). isalnum('&')=False
isalpha()	This function is used to test whether the given character is alphabetic character or not.	isalpha('a')=True(1) isalpha('5')=False(0)
isdigit()	This function is used to check whether or not it is a digit(0 to 9).	isdigit('9')=True isdigit('a')=False
islower()	This function is used to check whether the given character is lower case or not(a to z).	islower('a')=True islower('A')=False
isupper()	This function is used to check whether the given character is uppercase letter or not(A to Z).	isupper('a')=False isupper('A')=True

Writing a Character

- To write one character at a time to the terminal “**putchar()**” function is used in C.
- Syntax:

```
putchar(var_name);
```

- E.g:

```
char answer='Y';  
putchar(answer);
```

```
putchar('\n');
```

Formatted Input

- **scanf()** function is used to take the user input in various ways.
- Syntax:

scanf (“control string”, arg1, arg2, ..., argn);

- Control string specifies the format in which data has to be entered.
- %d ← Signed / Unsigned int
- %ld ← Long int
- %hd ← Short int

Inputting Integer Numbers

- Syntax:

```
scanf( "%wd1 %wd2 %wd3 ...", &var1, &var2, &var3,... );
```

- E.g:

```
scanf( "%2d %5d %d",&num1, &num2, &num3 );
```

- Case1: 50 32651 888
- Case2: 32651 50 888

```
scanf( "%d %*d %d",&data1,data2 );
```

- Case3: 123 456 789

	Case1	Case2
	num1	num1
	50	32
	num2	num2
	32651	651
	num3	num3
	888	50

Case3
data1
123
data2
789

Inputting Real Numbers

- Unlike int numbers, width of real numbers can not be specified.
- Float ← %f
- double ← %lf
- E.g:
- **scanf (“%f %f %f”, &x, &y, &z);**
- Case1: 123.45 4425.98E-2 66
- You can skip reading a real number using %*f specification.

Case1

x
123.45
y
44.2598
z
66.0

Inputting Character Strings

- For single character ← `getchar();`
- For multiple characters ← `scanf("%s",var);`
- **%wc** or **%ws** is used to specify number of characters while scanning string.
- Some other specifications of **scanf** function is as following:
- **%[characters]** ← Only allows specified characters
- **%[^characters]** ← Exactly reverse of above

Reading Blank Spaces

- Specifier %s can not be used with strings having blank spaces.
- But one can include white spaces in scanning using %[] specification.

```
main()
{
    char address[80];
    printf("Enter address\n");
    scanf("%[^\n]", address);
    printf("%-80s", address);
}
```

Output

```
Enter address
New Delhi 110 002
New Delhi 110 002
```

Error Detection in Input

- When a **scanf()** function completes reading list, it returns the value for every successful read.

E.g.: `scanf(“%d %f %s”, &a,&b,name);`

- Above statement will return value 3 with following value assignment.

20 150.25 motor



Error Detection in Input

```
main()
{
    int a;
    float b;
    char c;
    printf("Enter values of a, b and c\n");
    if (scanf("%d %f %c", &a, &b, &c) == 3)
        printf("a = %d b = %f c = %c\n", a, b, c);
    else
        printf("Error in input.\n");
}
```

```
Enter values of a, b and c
12 3.45 A
a = 12    b = 3.450000    c = A
Enter values of a, b and c
23 78 9
a = 23    b = 78.000000    c = 9
Enter values of a, b and c
8 A 5.25
Error in input.
Enter values of a, b and c
Y 12 67
Error in input.
Enter values of a, b and c
15.75 23 X
a = 15    b = 0.750000    c = 2
```

scanf Format Codes

Code	Format
%c	character
%s	character string
%d	decimal integers
%i	decimal integers
%f	floating pointer number (printf)
%lf	floating pointer number (scanf)
%e	scientific notation (with lower case e)
%E	scientific notation (with upper case E)
%g	the shorter of %f and %e
%G	the shorter of %f and %E
%u	unsigned decimal integer
%o	unsigned octal number
%x	unsigned hexadecimal number (with lower case letters)
%X	unsigned hexadecimal number (with upper case letters)
%p	pointer
%%	print %

Formatted Output

- **printf()** function is used to print output on the console screen.
- Syntax:

printf("control string", arg1,arg2, ... , argn);

- Similar to scanf() function, format specifier is used inside control string to tell compiler about variables being printed.

Output of Integer Numbers

- Syntax:

`printf(" %nd ", var_name);`

- Here, n specifies minimum field width for output.

<u><i>Format</i></u>	<u><i>Output</i></u>							
printf(“ %d ”,1234);	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td></tr></table>	1	2	3	4			
1	2	3	4					
printf(“ %7d ”,1234);	<table><tr><td></td><td></td><td></td><td>1</td><td>2</td><td>3</td><td>4</td></tr></table>				1	2	3	4
			1	2	3	4		
printf(“ %2d ”,1234);	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td></tr></table>	1	2	3	4			
1	2	3	4					
printf(“ %-7d ”,1234);	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td></td><td></td><td></td></tr></table>	1	2	3	4			
1	2	3	4					
printf(“ %07d ”,1234);	<table><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr></table>	0	0	0	1	2	3	4
0	0	0	1	2	3	4		

Output of Real Numbers

- Syntax:

`printf(" %f ", var_name);` OR

`printf(" %e ", var_name);`

- Here, **n** specifies minimum field width for output and **m** specifies decimal digits after floating point notation.
- We can also decide width and precision at runtime using variables.
- **`printf("%*.*f", width, precision, number);`**

Output of Real Numbers

Format

`printf("%7.4f",y)`

`printf("%7.2f",y)`

`printf("%-7.2f",y)`

`printf("%f",y)`

`printf("%10.2e",y)`

`printf("%11.4e",-y)`

`printf("%-10.2e",y)`

`printf("%e",y)`

Output

9	8	.	7	6	5	4
---	---	---	---	---	---	---

		9	8	.	7	7
--	--	---	---	---	---	---

9	8	.	7	7		
---	---	---	---	---	--	--

9	8	.	7	6	5	4
---	---	---	---	---	---	---

		9	.	8	8	e	+	0	1
--	--	---	---	---	---	---	---	---	---

-	9	.	8	7	6	5	e	+	0	1
---	---	---	---	---	---	---	---	---	---	---

9	.	8	8	e	+	0	1		
---	---	---	---	---	---	---	---	--	--

9	.	8	7	6	5	4	0	e	+	0	1
---	---	---	---	---	---	---	---	---	---	---	---

Output Format Codes & Flags

<i>specifier</i>	<i>argument</i>
d or i	<i>Signed INT</i>
u	<i>Unsigned INT</i>
o	<i>Unsigned Octal</i>
x or X	<i>Unsigned Hexadecimal (lower/upper case)</i>
e or E	<i>Floating Point</i>
g or G	<i>Shortest Representation</i>
a or A	<i>Hexadecimal Floating Point (lower/upper case)</i>
c	<i>Character</i>
s	<i>String of characters</i>
p	<i>Pointer address</i>

<i>flag</i>	<i>description</i>
-	<i>Left-justify (default: right)</i>
+	<i>Force print + sign with positives</i>
(space)	<i>Add space, if no sign before value</i>
#	<i>Precede o, x, or X with 0</i>
0	<i>Left pad number with zeros</i>

String I/O

- String is nothing but the collection of characters.
- “**gets()**” and “**puts()**” functions are used to take user input in string form.
- Syntax:

gets(string_var)

puts(string_var);

- Alternative Representation:

scanf(“%s”,string_var);

printf(“%s”,string_var);

Printing of Single Character

- Use the following format to print the single character using following format:

%wc

- The character will be **right justified** in **width of w** columns.

Common Output Flags

<i>Flag</i>	<i>Meaning</i>
-	Output is left-justified within the field. Remaining field will be blank.
+	+ or - will precede the signed numeric item.
0	Causes leading zeros to appear.
# (with o or x)	Causes octal and hex items to be preceded by O and Ox, respectively.
# (with e, f or g)	Causes a decimal point to be present in all floating point numbers, even if it is whole number. Also prevents the truncation of trailing zeros in g-type conversion.

Enhancing Readability of Output

- Correctness and clarity in output is very important.
- Some common measures are as follow:
 - Provide enough blank space between two numbers.
 - Introduce appropriate headings and variable names in output.
 - Print special extra messages whenever a peculiar condition occurs in the output.
 - Introduce extra blank lines between important sections of output.

Thank You